T1 极长颜色段（color）题解

2026 赛季 CSP-S 复赛热身赛（03） · 时间/内存：1.0 s / 512 MiB · 模数 10^9+7

0. 题目要算什么

给一个 01 串，把"连续相同的一段"合并后剩下的段数，就是该串的**极长颜色段数** $g(S)$。

> **示例：** `0 0 1 0 1 1` → 分成 `"00"` | `"1"` | `"0"` | `"11"` 共 4 段，所以 $g(001011) = 4$

题目要求：把所有"恰好 n 个 0、m 个 1"的串全部枚举，将它们的 $g(S)$ 求和，记为 $f(n,m)$，输出 $f(n,m) \bmod (10^9+7)$。

样例核对（$n=m=2$，共 6 个串）：

串 S	0011	0101	0110	1001	1010	1100	合计
$g(S)$	2	4	3	3	4	2	**18**

数据范围： $T \le 10^6$ 组询问，$0 \le n+m \le 2\times10^6$，$0 \le n, m \le 2\times10^6$。

1. 第一档：暴力枚举（15 分）

适用范围： subtask 1，$n+m \le 20$，$T \le 10$。

思路： 用 DFS 逐位决定填 0 还是 1，同时维护已用 0/1 的个数、上一位字符、当前段数。填每一位时判断：是第一位就直接新开一段，否则看和上一位是否不同——不同才新开一段。

复杂度： $O(2^L \cdot L)$，$L \le 20$ 时约 2×10^7，可以接受。

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;

int n, m, L;
long long ans;

// 逐位生成所有 01 串
// pos  = 当前要填的位置（从 0 开始）
```

```
// c0   = 已经用掉的 0 的个数
// c1   = 已经用掉的 1 的个数
// last = 上一位填的字符（填第一位时无意义）
// g    = 当前已经分出的段数
void dfs(int pos, int c0, int c1, int last, int g) {
    if (pos == L) {
        ans = (ans + g) % MOD;   // 整个串填完了，把段数加到答案里
        return;
    }

    // 情况一：这一位填 0（前提是 0 还没用完）
    if (c0 < n) {
        int ng;                  // 填完这一位后的新段数
        if (pos == 0) {
            ng = g + 1;          // 第一位，必定新开一段
        } else if (last != 0) {
            ng = g + 1;          // 和上一位不同，新开一段
        } else {
            ng = g;              // 和上一位相同，接在原段里
       }
        dfs(pos + 1, c0 + 1, c1, 0, ng);
    }

    // 情况二：这一位填 1（前提是 1 还没用完）
    if (c1 < m) {
        int ng;
        if (pos == 0) {
            ng = g + 1;
        } else if (last != 1) {
            ng = g + 1;
        } else {
            ng = g;
       }
        dfs(pos + 1, c0, c1 + 1, 1, ng);
    }
}

int main() {
    freopen("color.in", "r", stdin);
    freopen("color.out", "w", stdout);
```

```
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);

    int T;
    cin >> T;
    while (T--) {
        cin >> n >> m;
        L = n + m;
        if (L == 0) {
            cout << 0 << '\n';
            continue;            // 空串：段数为 0
       }
        ans = 0;
        dfs(0, 0, 0, -1, 0);
        cout << ans << '\n';
   }
    return 0;
}
```

为什么 $n+m=4000$ 就不行了？2^{4000} 是天文数字——必须换"数数"而非"造串"。

2. 第二档：递推 DP（40 分）

适用范围：subtask 2，$n+m \le 4000$。

思路：不造串，而是找 $f(n,m)$ 和"更小的 f"之间的递推关系。

按最后一位分成两类

所有"n 个 0、m 个 1"的串，按最后一位只能分成两类：**以 0 结尾**和**以 1 结尾**。分别算出这两类的段数之和，再加起来就是 $f(n,m)$。

第一类：以 0 结尾的串

任何一个以 0 结尾的串 S，都可以写成 $S = S' + "0"$，其中 S' 是一个"n−1 个 0、m 个 1"的串（把末尾那个 0 去掉即可）。
反过来，任取一个"n−1 个 0、m 个 1"的串 S'，末尾添一个 0 就得到 S——两者**一一对应**，不重不漏。

那么段数怎么变？看 S' 的最后一位是什么：

- 若 S' 以 1 结尾，则 $S = "\ldots10"$，末尾的 0 和前面的 1 不同 → 多出一段，$g(S) = g(S') + 1$；
- 若 S' 以 0 结尾，则 $S = "\ldots00"$，末尾的 0 和前面的 0 相同 → 接在原段里，$g(S) = g(S')$。

把这一类所有串的段数加起来：

$$\Sigma\, g(S) = \Sigma\, g(S') + (S' \text{ 以 } 1 \text{ 结尾的个数})$$

这两项分别是多少？

- $\Sigma\, g(S')$：S' 跑遍所有"n−1 个 0、m 个 1"的串，而**这正好就是 $f(n-1, m)$ 的定义**；
- "S' 以 1 结尾"的个数：S' 长度是 $L-1$，末位固定为 1，那么前 $L-2$ 位要放 $n-1$ 个 0、$m-1$ 个 1。从 $L-2$ 个位置里选 $n-1$ 个放 0 → 共 $C(L-2, n-1)$ 个。

所以第一类贡献 = $f(n-1, m) + C(L-2, n-1)$。

第二类：以 1 结尾的串

完全对称：$S = S' + "1"$，其中 S' 是"n 个 0、m−1 个 1"的串。当 S' 以 0 结尾时多出一段，否则段数不变。同理可得第二类贡献 = $f(n, m-1) + C(L-2, n-1)$。

两类相加得到递推式

$$f(n, m) = f(n-1, m) + f(n, m-1) + 2 \cdot C(L-2, n-1)$$

即 $f(n, m) = f(n-1, m) + f(n, m-1) + 2 \cdot C(n+m-2, n-1)$

边界： $f(0,0)=0$；$f(n,0)=f(0,m)=1$（只有一种颜色时串唯一，段数为 1）。

> **小例子验证：** 取 $n=m=1$，串只有 `01` 和 `10`，段数都是 2，所以 $f(1,1)=4$。代入递推式：$f(0,1) + f(1,0) + 2 \cdot C(0,0) = 1 + 1 + 2 \cdot 1 = 4$ ✓

复杂度： 填一张 ⩽ 4001 × 4001 的表，约 1.6×10^7 次运算，内存约 64 MB，稳过。

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;
const int MAXL = 4000;
```

```
int f[MAXL + 1][MAXL + 1];   // f[n][m]: n个0、m个1的所有串的g(S)之和

long long jc[MAXL + 5], ijc[MAXL + 5];

long long qpow(long long b, long long p) {
    long long res = 1;
    while (p) {
        if (p & 1) res = res * b % MOD;
        b = b * b % MOD;
        p >>= 1;
   }
    return res;
}

long long C(int x, int y) {
   // 组合数，0 <= y <= x <= 4000
    if (y < 0 || y > x) return 0;
    return jc[x] * ijc[y] % MOD * ijc[x - y] % MOD;
}

int main() {
    freopen("color.in", "r", stdin);
    freopen("color.out", "w", stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);

   // 预处理阶乘与逆元，用于 O(1) 取组合数
    jc[0] = 1;
    for (int i = 1; i <= MAXL; i++)
        jc[i] = jc[i - 1] * i % MOD;
    ijc[MAXL] = qpow(jc[MAXL], MOD - 2);
    for (int i = MAXL; i >= 1; i--)
        ijc[i - 1] = ijc[i] * i % MOD;

    int T;
    cin >> T;
    vector<pair<int, int>> q(T);
    int mxn = 0, mxm = 0;
```

```
    for (int i = 0; i < T; i++) {
        cin >> q[i].first >> q[i].second;
        mxn = max(mxn, q[i].first);
        mxm = max(mxm, q[i].second);
    }

    // DP: f(n,m) = f(n-1,m) + f(n,m-1) + 2*C(n+m-2, n-1)
    for (int n = 0; n <= mxn; n++) {
        for (int m = 0; m <= mxm; m++) {
            if (n == 0 && m == 0)
                f[n][m] = 0;           // 空串
            else if (n == 0 || m == 0)
                f[n][m] = 1;           // 只有一种颜色，段数恒为1
            else
                f[n][m] = (f[n - 1][m] + f[n][m - 1]
                    + 2LL * C(n + m - 2, n - 1)) % MOD;
       }
    }

    for (int i = 0; i < T; i++)
        cout << f[q[i].first][q[i].second] << '\n';
    return 0;
}
```

> **可选优化：**递推只依赖"上一行"和"本行左侧"，可压成一维滚动数组（从右往左更新），内存降至 $O(m)$。

3. 破题关键：从"数串"到"数断点"

前两档都在"一个一个串地数"，效率被串的个数卡住。这一节换个视角：**直接把答案写成一个公式**。

第一步 段数 = 1 + 断点数

"断点"指**相邻两位字符不同**的位置。沿串从左到右走：第一段记为 1；之后每遇到一个断点就进入新段，+1。

> **示例：** `001011` 逐对检查 —— `0|0` 同、`0|1` 异、`1|0` 异、`0|1` 异、`1|1` 同，共 3 个断点，所以 $g = 1 + 3 = 4$。

关键：段数是全局性质（要看完整串才能数出段），断点却是局部性质（只看相邻两位即可判断）。局部性质可以逐位独立统计——这是能推出公式的根本原因。

第二步 把总和拆成两块

设 $L = n + m$，把每个串贡献的 1 与"断点数"拆开：

$$f(n, m) = (\text{串的总数}) + (\text{所有串的断点数之和})$$

- **串的总数：**L 个位置里选 n 个放 0 → $C(L, n)$；
- **所有串的断点数之和：**难点，见第三步。

第三步 断点总数：一个位置一个位置地数

直接数"每个串有几个断点"很困难，因为不同串的断点数相差很大。换个思路：**按断点所在的位置来数**。

① **断点有两类。**相邻两位不同，只可能是 `0|1` 或 `1|0` 这两种。

② **断点能出现在几个位置？**长度为 $L = n+m$ 的串共有 $L-1$ 对相邻位，所以断点一共有 $L-1 = n+m-1$ 个可能的位置。

③ **固定一个位置，算它的贡献。**把断点"钉"在位置 i 上——这等于**确定了两个格子的内容**（第 i 位和第 $i+1$ 位必须是一个 0 和一个 1）。既然已经用掉了一个 0 和一个 1，剩下的 $L-2$ 个格子要放 $n-1$ 个 0、$m-1$ 个 1：

- 填 `0|1`：剩下 $L-2$ 位选 $n-1$ 个放 0 → $C(L-2, n-1)$ 种；
- 填 `1|0`：同理 → 也是 $C(L-2, n-1)$ 种。

于是**位置 i 的贡献**（即"在位置 i 处是断点"的串有多少个）$= 2 \cdot C(L-2, n-1)$。

> **这个贡献与 i 无关**——它只取决于"用掉了一个 0 和一个 1"，跟断点落在哪个位置毫无关系。所以 $n+m-1$ 个位置的贡献全都一样，不必逐个重算。

④ **把所有位置的贡献加起来。**为什么这样加出来正好是"所有串的断点数之和"？因为**每一个断点都恰好落在某一个位置上**——按位置累加，就是把所有串的所有断点不重不漏地数了一遍。

$$\text{所有串的断点数之和} = (L-1)\cdot 2\cdot C(L-2,\,n-1)$$

即 $(n+m-1)\cdot 2\cdot C(n+m-2,\,n-1)$

第四步 合并得到公式

$$f(n,\,m) = C(L,\,n) + 2\cdot(L-1)\cdot C(L-2,\,n-1)$$

其中 $L = n + m$，要求 $n \ge 1$ 且 $m \ge 1$

边界必须先特判： $n = m = 0$ 时 $f = 0$；$n = 0$ 或 $m = 0$ 但 $L \ge 1$ 时 $f = 1$。若不特判，$C(L-2,\,n-1)$ 的下标 $n-1$ 会变成 −1，组合数无定义、代码会越界。

验算： $n = m = 2$，$L = 4$：$C(4,2) = 6$，$2 \times 3 \times C(2,1) = 12$，合计 **18** ✓

答案已经到手，剩下唯一的问题是**组合数怎么算得足够快**——这正是下一节正解要做的事。

4. 正解：预处理阶乘，O(1) 组合数（100 分）

适用范围： subtask 3~4，$T \le 10^6$，$n+m \le 2\times10^6$。

上一节已经推导出闭式公式，剩下的问题只是**如何快速算组合数**。当 T 很大时，每次询问都必须 $O(1)$ 完成。

关键观察： 无论 T 多大，所有询问的 $L = n+m \le 2\times10^6$，组合数中 x 最大也不超过 2×10^6。

因此只需一次 $O(2\times10^6)$ 预处理，算出 $0!$ ~ $(2\times10^6)!$ 的阶乘表，之后每次取组合数都是 $O(1)$：

$$C(x,\,y) = jc[x]\cdot inv(jc[y])\cdot inv(jc[x-y])\ \ (mod\ MOD)$$

逆元通过费马小定理 + 快速幂求出：$inv(a) = a^{MOD-2}$（因模数 10^9+7 是质数）。

总复杂度： $O(2\times10^6 + T\cdot \log MOD)$。$T = 10^6$ 时约 2×10^8 次乘法，实测最难点约 0.75 秒。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll Mod = 1e9 + 7;
const int N = 2e6 + 5;
```

```
ll T, n, m, jc[N];

// 费马小定理：快速幂求乘法逆元
ll qpow(ll b, ll p) {
    ll res = 1;
    while (p) {
        if (p % 2 == 1)
            res = res * b % Mod;
        b = b * b % Mod;
        p /= 2;
   }
    return res;
}

// O(1) 组合数（依赖预处理的阶乘）
ll C(ll x, ll y) {
    if (y == 0) return 1;
    return jc[x] * qpow(jc[x - y], Mod - 2) % Mod
         * qpow(jc[y], Mod - 2) % Mod;
}

// 核心公式：f(n,m) = C(L,n) + 2*(L-1)*C(L-2,n-1)
ll f(ll x, ll y) {
    return (C(x + y, x)
        + (x + y - 1) * C(x + y - 2, x - 1) % Mod
        * 2 % Mod) % Mod;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);

   // 预处理阶乘
    jc[0] = 1;
    for (int i = 1; i < N; i++)
        jc[i] = jc[i - 1] * i % Mod;

    cin >> T;
```

```
    while (T--) {
        cin >> n >> m;
        if (n + m == 0)
            cout << 0 << '\n';
        else if (n == 0 || m == 0)
            cout << 1 << '\n';
        else
            cout << f(n, m) << '\n';
    }
    return 0;
}
```

代码逐段解析

- `jc[i]`：预处理 $i!$，主函数开头一个循环完成（数组开 `2e6+5`，因为 L 最大 2×10^6）。
- `qpow(b, p)`：快速幂，$O(\log p)$ 算 b^p；配合费马小定理，`qpow(a, MOD-2)` 即 a 的逆元。
- `C(x, y)`：$jc[x]\cdot jc[y]^{-1}\cdot jc[x-y]^{-1}$；`if(y==0) return 1` 是边界剪枝。
- `f(x, y)`：核心公式 $C(L,n)+2(L-1)C(L-2,n-1)$ 的直接实现。
- `main`：先特判两个边界（$n=m=0$ 输出 0；n 或 m 为 0 输出 1），其余交给 `f()`。

> **进一步加速（可选）：**当前每个询问调用 4 次快速幂（`f` 里两个组合数各需 2 次）。其实**逆阶乘也能线性预处理**：先算 `inv(jc[N])` 一次，再用 `ijc[i-1] = ijc[i] · i` 前推。改完后每次询问纯 $O(1)$，$T=10^6$ 时更稳、更不容易卡常。（写法参见第 2 节 DP 代码中的 `ijc` 数组）

5. 易错点清单

1. $n=m=0$ 时答案是 **0**，不是 1——空串没有段。
2. $n=0$ 或 $m=0$ 且 $L\ge1$ 时答案是 **1**——串唯一，整串就是一段。这两个边界必须先特判，否则公式中会出现负数下标导致越界。
3. 阶乘数组要开到 $2\times10^6+5$，因为 $L=n+m$ 最大为 2×10^6；组合数函数中记得判 $y<0$ 或 $y>x$ 时返回 0。
4. 所有乘法用 `long long`：$10^9\times10^9$ 会爆 `int`。
5. $T=10^6$ 时输出量很大（约 10^6 行），务必关闭 `cin` 同步（代码已做）；想更快可用 `string` 拼接后一次性输出。
6. 模数 10^9+7 是质数，费马小定理才成立；更换模数时需先确认其是否为质数。

6. 一句话总结

段数 = 1 + 断点数——把"全局的段"转化为"局部的断点"，再按位置统计每个断点位置的贡献，一步推出闭式公式。

三档做法本质上是同一思路的三种强度：**把串真的造出来**（15 分）→ **只记递推关系**（40 分）→ **推出闭式公式、再预处理组合数**（100 分）。
